

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»  
Институт перспективной инженерии  
Департамент цифровых робототехнических систем и электроники

Отчёт к лабораторной работе №1

Тема: «Пример обратного выполнения для программы вычисления  
степени числа  $x$

Дисциплина: «Тестирование и отладка программного обеспечения»

**Выполнил:**

Студент группы ПИЖ-б-о-23-1  
направление подготовки:  
09.03.04 «Программная  
инженерия»

Астраков Борис Александрович

---

**Проверил:**

Ассистент ДЦРСиЭ

Потапов Иван Романович

---

**Цель работы:** освоить метод тестирования обратного выполнения программы.

### Ход работы:

Реализуем программу вычисления степени числа на языке Python для дальнейшего тестирования обратного выполнения:

```
def power(x, n):
    """Вычисляет степень n числа x (n >= 1)."""
    z = 1
    i = 1
    while n >= i:
        z = z * x
        i = i + 1
    return z

if __name__ == "__main__":
    print(power(2, 3))
```

Реализуем для этого кода трассировку через запись в массив всех переменных и добавим отладочный вывод. Переменная *i* будет выступать счётчиком цикла, *z* будет фиксировать промежуточные состояния возведения в степень, а *x* и *n* — само число и его степень соответственно. Также пропишем проверку частных случаев (степень равна единице, степень нуля, степень единицы и другие):

```
def power_traced(x, n):
    """Функция возведения в степень с трассировкой всех переменных"""
    trace = []
    z = 1
    trace.append(("z=1", {"x": x, "n": n, "z": z}))

    i = 1
    trace.append(("i=1", {"x": x, "n": n, "z": z, "i": i}))

    while n >= i:
        trace.append((f"check n>=i ({n})>={i})", {"x": x, "n": n, "z": z, "i":
i}))
        z = z * x
        trace.append(("z=z*x", {"x": x, "n": n, "z": z, "i": i}))
        i = i + 1
        trace.append(("i=i+1", {"x": x, "n": n, "z": z, "i": i}))
```

```

        trace.append(("exit", {"x": x, "n": n, "z": z, "i": i}))
    return z, trace

def reverse_check(trace):
    """
    Обратное выполнение: идём снизу вверх и проверяем,
    что каждое предыдущее состояние выводится из текущего.
    """
    print("\n=== ОБРАТНОЕ ВЫПОЛНЕНИЕ (снизу вверх) ===")
    ok = True
    for idx in range(len(trace) - 1, 0, -1):
        op_curr, st_curr = trace[idx]
        op_prev, st_prev = trace[idx - 1]
        print(f"\nШаг {idx}: {op_curr} -> {st_curr}")
        print(f"    Проверяем предыдущее: {op_prev} -> {st_prev}")

        # Простейшая проверка: значения x и n не меняются
        if st_curr["x"] != st_prev["x"] or st_curr["n"] != st_prev["n"]:
            print("    ОШИБКА: x или n изменились!")
            ok = False

        # Проверка для z = z*x
        if op_curr == "z=z*x" and st_prev["x"] != 0:
            expected_prev_z = st_curr["z"] // st_prev["x"]
            if expected_prev_z != st_prev["z"]:
                print(f"    ОШИБКА: ожидалось z={expected_prev_z}, получено z={st_prev['z']}")
                ok = False
            else:
                print(f"    ОК: обратный ход z = {st_curr['z']}/{st_prev['x']} = {expected_prev_z}")

        # Проверка для i = i+1
        if op_curr == "i=i+1":
            if st_prev["i"] + 1 != st_curr["i"]:
                print(f"    ОШИБКА: i должно быть {st_curr['i']-1}, а равно {st_prev['i']}")
                ok = False
            else:
                print(f"    ОК: обратный ход i = {st_curr['i']} - 1 = {st_prev['i']}")

    print("\n=== РЕЗУЛЬТАТ ОБРАТНОЙ ПРОВЕРКИ ===")
    print("Все шаги согласованы" if ok else "Обнаружены ошибки!")
    return ok

# ----- ТЕСТОВЫЕ СЛУЧАИ (по методичке) -----
def run_tests():

```

```

test_cases = [
    # (x, n, ожидаемый результат, описание)
    (2, 3, 8, "Обычный случай: 2^3 = 8"),
    (2, 1, 2, "Минимальная степень: 2^1 = 2"),
    (5, 4, 625, "5^4 = 625"),
    (1, 100, 1, "1^100 = 1"),
    (0, 5, 0, "0^5 = 0"),
    (10, 2, 100, "10^2 = 100"),
]

print("=" * 60)
print("ТЕСТИРОВАНИЕ ФУНКЦИИ power(x, n)")
print("=" * 60)

all_passed = True
for x, n, expected, desc in test_cases:
    result = power(x, n)
    status = "PASS" if result == expected else "FAIL"
    if result != expected:
        all_passed = False
    print(f"[{status}] {desc}: power({x},{n}) = {result}, ожидалось {expected}")

print("\n" + "=" * 60)
print("ОБРАТНОЕ ТЕСТИРОВАНИЕ (на примере x=2, n=3)")
print("=" * 60)

result, trace = power_traced(2, 3)
reverse_check(trace)

return all_passed

if __name__ == "__main__":
    run_tests()

```

Получим следующий вывод в консоль:

```

=====
ТЕСТИРОВАНИЕ ФУНКЦИИ power(x, n)
=====
[PASS] Обычный случай: 2^3 = 8: power(2,3) = 8, ожидалось 8
[PASS] Минимальная степень: 2^1 = 2: power(2,1) = 2, ожидалось 2
[PASS] 5^4 = 625: power(5,4) = 625, ожидалось 625
[PASS] 1^100 = 1: power(1,100) = 1, ожидалось 1
[PASS] 0^5 = 0: power(0,5) = 0, ожидалось 0
[PASS] 10^2 = 100: power(10,2) = 100, ожидалось 100

```

=====

ОБРАТНОЕ ТЕСТИРОВАНИЕ (на примере  $x=2$ ,  $n=3$ )

=====

=== ОБРАТНОЕ ВЫПОЛНЕНИЕ (снизу вверх) ===

Шаг 11: exit -> {'x': 2, 'n': 3, 'z': 8, 'i': 4}

Проверяем предыдущее:  $i=i+1$  -> {'x': 2, 'n': 3, 'z': 8, 'i': 4}

Шаг 10:  $i=i+1$  -> {'x': 2, 'n': 3, 'z': 8, 'i': 4}

Проверяем предыдущее:  $z=z*x$  -> {'x': 2, 'n': 3, 'z': 8, 'i': 3}

ОК: обратный ход  $i = 4 - 1 = 3$

Шаг 9:  $z=z*x$  -> {'x': 2, 'n': 3, 'z': 8, 'i': 3}

Проверяем предыдущее: check  $n \geq i$  ( $3 \geq 3$ ) -> {'x': 2, 'n': 3, 'z': 4, 'i': 3}

ОК: обратный ход  $z = 8/2 = 4$

Шаг 8: check  $n \geq i$  ( $3 \geq 3$ ) -> {'x': 2, 'n': 3, 'z': 4, 'i': 3}

Проверяем предыдущее:  $i=i+1$  -> {'x': 2, 'n': 3, 'z': 4, 'i': 3}

Шаг 7:  $i=i+1$  -> {'x': 2, 'n': 3, 'z': 4, 'i': 3}

Проверяем предыдущее:  $z=z*x$  -> {'x': 2, 'n': 3, 'z': 4, 'i': 2}

ОК: обратный ход  $i = 3 - 1 = 2$

Шаг 6:  $z=z*x$  -> {'x': 2, 'n': 3, 'z': 4, 'i': 2}

Проверяем предыдущее: check  $n \geq i$  ( $3 \geq 2$ ) -> {'x': 2, 'n': 3, 'z': 2, 'i': 2}

ОК: обратный ход  $z = 4/2 = 2$

Шаг 5: check  $n \geq i$  ( $3 \geq 2$ ) -> {'x': 2, 'n': 3, 'z': 2, 'i': 2}

Проверяем предыдущее:  $i=i+1$  -> {'x': 2, 'n': 3, 'z': 2, 'i': 2}

Шаг 4:  $i=i+1$  -> {'x': 2, 'n': 3, 'z': 2, 'i': 2}

Проверяем предыдущее:  $z=z*x$  -> {'x': 2, 'n': 3, 'z': 2, 'i': 1}

ОК: обратный ход  $i = 2 - 1 = 1$

Шаг 3:  $z=z*x$  -> {'x': 2, 'n': 3, 'z': 2, 'i': 1}

Проверяем предыдущее: check  $n \geq i$  ( $3 \geq 1$ ) -> {'x': 2, 'n': 3, 'z': 1, 'i': 1}

ОК: обратный ход  $z = 2/2 = 1$

```
Шаг 2: check n>=i (3>=1)  -> {'x': 2, 'n': 3, 'z': 1, 'i': 1}
      Проверяем предыдущее: i=1 -> {'x': 2, 'n': 3, 'z': 1, 'i': 1}
```

```
Шаг 1: i=1  -> {'x': 2, 'n': 3, 'z': 1, 'i': 1}
      Проверяем предыдущее: z=1 -> {'x': 2, 'n': 3, 'z': 1}
```

=== РЕЗУЛЬТАТ ОБРАТНОЙ ПРОВЕРКИ ===

Все шаги согласованы

### **Вывод:**

Проделав работу, освоил метод тестирования обратного выполнения программы.